

Monthly Progress Report

Experimental Setup, MADS Integration and Data Visualization

Author: Fernando Cardenas Acosta

Supervisor: Anna Carla Araujo

Institution: INSA Toulouse, ICA, MFJA

Reporting period: July 2026

Date: July 29, 2026

Abstract

During the reporting period, the work focused on bringing the complete experimental acquisition chain into operation, developing a practical understanding of the MADS framework, implementing a graphical tool for rapidly inspecting data stored in the database, and performing measurements at the five operating speeds of the machine.

The experimental setup was connected to a Raspberry Pi, where a MADS source received the data provided by the acquisition system and published the corresponding measurements. The communication between the Raspberry Pi and the computer was configured and monitored using the MADS terminal user interface. The structure of the MADS configuration file and the organization of the stored database documents were also studied.

A graphical application, referred to in this report as the MADS Viewer, was developed to facilitate the selection, visualization and export of the recorded measurements. The tests performed with the machine showed that the selected current sensor was not suitable for reliably distinguishing the different operating conditions. Based on these results, the remaining work will focus on improving the measurement system, consolidating the MADS acquisition chain and repeating the experimental tests with a more suitable sensor.

Keywords: MADS, Raspberry Pi, data acquisition, MongoDB, data visualization, current measurement, experimental testing.

Contents

Abstract	1
1 Introduction	3
2 Commissioning of the Complete Experimental Setup	3
3 Learning and Practical Use of MADS	4
3.1 General Architecture	5
3.2 Understanding the <code>mad.s.ini</code> File	5
3.3 MADS Agents	6
3.4 Source Running on the Raspberry Pi	7
3.5 Connection Between the Raspberry Pi and the Computer	8
3.6 Plugins and Use of the MADS TUI	8
3.6.1 MADS Plugins	8
3.6.2 Use of the <code>tui.plugin</code>	8
3.7 Understanding the Database Structure	9
4 Development of the MADS Viewer	10
4.1 Motivation and Objectives	10

4.2	Main Functionalities	11
4.3	User Workflow	11
4.4	Implementation	12
5	Tests Performed	13
5.1	Experimental Procedure	13
5.2	Recorded Data	13
5.3	Limitations of the Signal Converter	15
6	Recommendations for the Measurement System	16
6.1	Recommended Current Clamp: SCT-013-060	16
6.2	Preferred Signal Conditioner: LeChacal CTC01	17
6.3	Alternative Integrated Solution: ElectroDragon Energy Monitoring Shield	17
6.4	Alternative Custom Conditioning Circuit	18
6.5	Recommended Implementation Sequence	18
7	Continuation of the Work	19
7.1	Improvement of the Measurement System	19
7.2	Consolidation of the MADS Acquisition Chain	19
7.3	Further Development of the MADS Viewer	20
7.4	Repetition of the Five-Speed Tests	20
7.5	Proposed Work Plan	20
8	Conclusion	21

1. Introduction

The general objective of the project is to develop an experimental data acquisition and analysis chain for studying the operation of a machine under different working conditions. The system must be able to acquire the measurements produced by the experimental setup, transmit them to a processing device, store them in a database and make them available for subsequent visualization and analysis.

The physical structure of the experimental setup had been assembled during an earlier phase of the project. The work carried out during the present reporting period therefore concentrated on bringing the complete chain into operation and validating the communication between its different elements.

Particular attention was given to the practical use of MADS. This included running a data source on the Raspberry Pi, configuring communication with the computer, monitoring the published information through the MADS terminal user interface, understanding the `mads.ini` configuration file and interpreting the structure of the data stored in the database.

A graphical interface was also developed to simplify the inspection of the recorded experiments. This interface provides rapid access to the database and allows the user to select, plot and export the relevant measurements without preparing a new analysis script for each test.

Finally, a set of experiments was performed at five operating speeds of the machine. These tests validated several elements of the acquisition and storage chain, but also revealed an important limitation: the selected current sensor was not suitable for obtaining sufficiently reliable measurements for the intended application.

The report is organized into six main technical sections:

- commissioning of the complete experimental setup;
- learning and practical use of MADS;
- development of the MADS Viewer;
- tests performed at the five machine speeds;
- evaluation of the limitations of the current sensor;
- continuation of the work.

2. Commissioning of the Complete Experimental Setup

The experimental structure used in this project had been assembled before the beginning of the present reporting period. Consequently, the objective was not to redesign the mechanical or electrical setup, but to put all of its components into operation as a complete acquisition chain.

The system included the machine under test, the current sensor, the acquisition electronics, the Raspberry Pi and the computer used for monitoring and data analysis. The output of the current sensor was acquired by the embedded acquisition system and transmitted to the Raspberry Pi through a serial connection.

The Raspberry Pi acted as the interface between the physical experiment and the MADS environment. It received the serial measurements, interpreted the transmitted values and published them through a MADS source. The resulting messages could then be monitored, stored

and accessed from the computer.

The main objective of the commissioning process was to verify the complete path followed by a measurement:

Machine → Current sensor → Acquisition system → Raspberry Pi → MADS →
Database → Visualization

The communication between each stage was tested progressively. First, the serial transmission between the acquisition system and the Raspberry Pi was verified. The MADS source was then used to publish the received values. Finally, the data were observed from the computer and their presence in the database was confirmed.



Figure 1: Complete experimental setup used for the acquisition tests.

The commissioning process confirmed that measurements could travel through the complete acquisition chain and be stored for later analysis. This result provided the foundation for the subsequent work on MADS, the MADS Viewer and the experimental tests.

3. Learning and Practical Use of MADS

A significant part of the work consisted of learning how to configure and operate MADS in a distributed acquisition system. The main objective was to understand the practical flow of information from the source running on the Raspberry Pi to the monitoring, storage and visualization tools available on the computer.

3.1. General Architecture

MADS uses independent agents that exchange information through a broker. In the configuration used for this project, the Raspberry Pi source published the experimental measurements, the logger stored the received messages in MongoDB, and the TUI was used to control the recording intervals and associate metadata with each test.

The broker, logger and sink formed part of the complete MADS architecture, although the main practical work during this period focused on configuring the application, operating the source on the Raspberry Pi and using the TUI during the experimental tests.

The general data flow was organized as follows:

Acquisition system $\longrightarrow$ Raspberry Pi source $\longrightarrow$ MADS broker $\longrightarrow$ Logger $\longrightarrow$ MongoDB

3.2. Understanding the `mads.ini` File

The `mads.ini` file is the main configuration file of a MADS application. It defines the agents that must be launched, the plugins used by those agents, their operating parameters and the way in which they communicate.

Understanding this file was one of the first necessary steps before operating the complete acquisition chain. It made it possible to identify how the different MADS components were connected and how their behaviour could be modified without changing the application source code.

The configuration file was used to define or modify:

- the agents included in the MADS application;
- the broker address used by the different devices;
- the plugins associated with each agent;
- the parameters of the source running on the Raspberry Pi;
- the serial communication settings;
- the names of the published or subscribed topics;
- the logger configuration;
- the database connection parameters.

This was particularly important because the MADS application was distributed between the Raspberry Pi and the computer. The broker address and the agent parameters had to be configured correctly so that the components running on both devices could participate in the same MADS network.

One of the main lessons was that the structure of the acquisition chain could be reorganized through the `mads.ini` file. Network addresses, topics, plugins and storage parameters could therefore be changed without modifying or recompiling the complete application.

Table 1 summarizes the most relevant groups of parameters.

Table 1: Main configuration elements studied in `mads.ini`.

Configuration element	Purpose
Agent definition	Specifies which MADS components must be executed.
Broker configuration	Defines the network address used for communication between agents.
Plugin selection	Associates each agent with the software component that provides its functionality.
Source parameters	Defines the serial communication and publication settings of the Raspberry Pi source.
Topic configuration	Determines where data are published and which messages are received.
Logger configuration	Defines when and where the experimental measurements are stored.
Database configuration	Specifies the connection parameters used for MongoDB storage.

3.3. MADS Agents

The basic building blocks of a MADS network are agents. An agent is an independent component that exchanges information with other agents through the broker. Depending on its role in the data flow, an agent can operate as a source, a filter or a sink.

A source agent produces data and publishes it to the MADS network. A filter agent receives data, processes it and publishes a new result. A sink agent receives data and performs an action without publishing a new measurement.

In the acquisition chain used for this project, the source running on the Raspberry Pi introduced the experimental measurements into the MADS network. The logger received and stored the published messages in MongoDB. Other sink agents could also receive the data for visualization or additional processing.

The broker was responsible for dispatching the messages exchanged by the agents. The agents did not need to establish direct connections with one another, as they only needed to connect to the same broker.

Table 2 briefly summarizes the main roles involved in the acquisition chain.

Table 2: Main MADS agent roles involved in the acquisition chain.

Element	Role
Broker	Dispatches messages between the agents connected to the MADS network.
Source agent	Produces and publishes the experimental measurements.
Filter agent	Receives and processes data before publishing a new result.
Logger	Receives and stores the published measurements in MongoDB.
Sink agent	Receives published data for visualization or another final action.

3.4. Source Running on the Raspberry Pi

The MADS source running on the Raspberry Pi represented the entry point of the experimental measurements into the MADS network. Its main function was to receive the information transmitted by the acquisition system through the serial connection and publish the corresponding data in a structured format.

The source agent executed the serial-reading logic required by the application. It had to maintain communication with the acquisition device while also publishing the received measurements through MADS.

Its operation involved several stages:

1. opening and configuring the serial connection;
2. receiving the measurements transmitted by the acquisition system;
3. separating and interpreting the different data fields;
4. associating the received values with their variable names;
5. publishing the resulting message through the MADS network;
6. repeating the operation continuously during the experiment.

The published messages contained the current measurement and additional information used to identify and organize the samples. The main fields are summarized in Table 3.

Table 3: Main fields published by the source agent.

Field	Type or unit	Description
n	Integer	Sequential sample number used to follow the continuity of the acquisition.
t_us	Microseconds	Time provided by the Arduino acquisition system for the corresponding measurement.
I_mA	Milliamperes	Current value obtained from the measurement system.
timestamp	Date and time	Time associated with the reception and storage of the message.

The sample number was useful for inspecting the continuity of the data stream, while **t_us** provided the time reference generated by the Arduino acquisition system.

For each test, the documents corresponding to the relevant experimental interval were first selected using the mark-in and mark-out timestamps recorded through the MADS TUI. Once the appropriate data segment had been extracted from the database, the Arduino time expressed in microseconds was used to construct the horizontal axis and ensure a reliable and consistent temporal representation of the measurements.

Understanding the distinction between the database timestamp and **t_us** was therefore essential for both selecting the correct documents and accurately plotting the experiments.

3.5. Connection Between the Raspberry Pi and the Computer

The acquisition system was distributed between the Raspberry Pi and the computer. The Raspberry Pi received the physical measurements and executed the source agent, while the computer was used to configure the application, control the experimental recording, access the database and run the graphical visualization interface.

Both devices had to communicate through the same network. This required identifying the Raspberry Pi network address, verifying the connection between the devices and configuring the agents with the correct broker address in the `mads.ini` file.

The communication was validated by confirming that the messages published by the source agent on the Raspberry Pi were received and stored by the MADS components running on the computer. This demonstrated that the acquisition and data-management tasks could be distributed between separate devices.

Remote access to the Raspberry Pi was also used during configuration and testing. This made it possible to launch the source agent, inspect its execution and modify the required parameters without directly connecting a dedicated screen or keyboard to the Raspberry Pi.

A time-synchronization configuration was also introduced on the Raspberry Pi. As soon as the Raspberry Pi connected to the computer, it obtained the computer time. This ensured that the timestamps generated on both devices were aligned, which was necessary for associating the metadata intervals with the corresponding measurement documents.

3.6. Plugins and Use of the MADS TUI

3.6.1. MADS Plugins

Plugins allow general-purpose MADS agents to be adapted to different applications. For example, a source agent can load a plugin that reads a sensor, receives serial data or provides an interactive terminal interface. Similarly, a sink agent can load a plugin that stores, displays or exports the messages received from the network.

The plugin to be loaded, together with its operating parameters, is defined in the `mads.ini` configuration file. This modular structure makes it possible to change the functionality of an agent without redesigning the complete MADS network.

3.6.2. Use of the `tui.plugin`

The MADS terminal user interface was used to control the recording of the experimental data. In particular, it allowed the logger to be activated only during the relevant test intervals and deactivated when no useful measurements were being produced.

This prevented unnecessary data from being stored in the database and reduced the amount of information that had to be processed afterwards.

The TUI was also used to introduce metadata associated with each experiment. Mark-in and mark-out timestamps were recorded at the beginning and end of every test, allowing the corresponding measurement documents to be identified and extracted from the database.

Additional metadata fields were used to describe the operating condition of the machine, including the rotational speed associated with each test.

The TUI was therefore used to:

- modify the metadata associated with each experimental condition;
- activate the logger at the beginning of each test;
- record the mark-in timestamp;
- record the mark-out timestamp at the end of the test;
- deactivate the logger after the relevant acquisition interval;
- avoid storing unnecessary measurements;
- facilitate the identification of the documents corresponding to each test.

This procedure was essential for organizing the experimental datasets. The timestamps and metadata published using the TUI made it possible to locate the relevant database documents and associate them with the correct machine speed and experimental conditions.

The screenshot shows a terminal window titled 'fcardenas-ac@insa-09082: ~'. The interface has a menu bar with 'Fichier', 'Édition', 'Affichage', 'Recherche', 'Terminal', and 'Aide'. The main area is a form with the following sections:

Machine operator	Operator name	
MADS user	MADS user	
Operation	Operation description	
Trial number	1	
Comment	No comment	
Logging	Enabled Disabled	Logging
Tags	☐ This is a test ☐ Custom tag 1 ☐ Custom tag 2 ☐ Custom tag 3	
Mark In Mark Stop Restart		

Figure 2: MADS TUI used to control the logger, define the mark-in and mark-out timestamps, and introduce the metadata associated with each experimental test.

3.7. Understanding the Database Structure

The final stage of the MADS acquisition chain was the storage of the measurements in MongoDB. Understanding how the logger organized the recorded information was necessary before the data could be correctly selected, visualized and exported.

While the logger was active, all the messages received from the source were stored in the same MongoDB collection. Each document corresponded to one acquired sample and contained the measured current, the sequential sample number, the acquisition time generated by the Arduino and additional fields added by the MADS environment.

A simplified example of a stored measurement document is shown below.

```
1 {  
2   "I_mA": 13340.54,  
3   "n": 4820467,  
4   "t_us": 2627503520,  
5   "agent_id": "",  
6   "hostname": "mads-pi-01",  
7   "timestamp": {  
8     "$date": "..."  
9   }  
10 }
```

Listing 1: Simplified structure of a stored measurement document.

The experimental metadata introduced through the MADS TUI were stored separately in another MongoDB collection. These metadata documents contained the information required to identify each test, including its operating condition and the mark-in and mark-out timestamps defining the relevant acquisition interval.

The database therefore contained two complementary types of information:

- measurement documents stored continuously by the logger while it was active;
- metadata documents generated through the TUI to identify and delimit each experiment.

The measurement documents included both the Arduino time, expressed in microseconds through `t_us`, and the timestamp associated with the Raspberry Pi. A clock-synchronization configuration was added to the Raspberry Pi so that, as soon as it connected to the computer, it adopted the computer time. As a result, the timestamps generated by both devices were aligned and could be used consistently when relating the metadata intervals to the stored measurements.

This organization was directly used by the MADS Viewer. The mark-in and mark-out timestamps stored in the metadata collection were used to select the documents corresponding to each test from the measurement collection. The Arduino microsecond field was then used to construct a precise and reliable time axis for plotting the selected data.

4. Development of the MADS Viewer

Once the acquisition chain was operational, a tool was required to inspect the recorded measurements efficiently. Accessing the database manually or preparing a dedicated script for every experiment would have made the analysis process unnecessarily slow.

The MADS Viewer was therefore developed as a graphical interface for accessing and plotting the data stored in the database. Its objective was to provide a rapid first inspection of the experiments and to simplify the preparation of figures and datasets for subsequent analysis.

4.1. Motivation and Objectives

The experiments generated collections containing a large number of samples. Before performing a detailed analysis, it was necessary to verify that the data had been correctly recorded, identify the available variables and select the relevant time intervals.

The main objectives of the viewer were:

- provide direct access to the experimental database;
- display the available datasets and variables;
- allow the user to select a measurement interval;
- graph the selected variable rapidly;
- display the corresponding numerical values;
- calculate basic descriptive statistics;
- export the selected data;
- export the generated figure.

The application does not replace a complete scientific analysis. Instead, it serves as an intermediate tool between data acquisition and detailed processing.

4.2. Main Functionalities

The viewer connects to MongoDB and retrieves the information required for the graphical interface. The user can select the database, collection, variable and time interval corresponding to the experiment of interest.

Table 4 summarizes the main functions implemented.

Table 4: Main functions of the MADS Viewer.

Function	Purpose
Database connection	Access the measurements stored by the MADS acquisition chain.
Collection selection	Select a specific experiment or group of measurements.
Variable selection	Choose the measurement that must be displayed.
Interval filtering	Restrict the analysis to a selected range of samples or time.
Graphical visualization	Inspect the evolution of the selected variable.
Table view	Examine the individual numerical values.
Statistical summary	Calculate basic indicators for the selected data.
CSV export	Save the selected measurements for additional analysis.
PNG export	Save the generated graph for reports or presentations.

4.3. User Workflow

The typical workflow of the MADS Viewer is the following:

1. connect to the MongoDB server;
2. select the required database;
3. choose the collection corresponding to an experiment;
4. select the time and measurement variables;

5. define the desired data interval;
6. load and display the selected values;
7. inspect the graph and the corresponding table;
8. export the selected data or figure when required.

This workflow makes it possible to move rapidly from a stored experiment to a usable graphical representation.

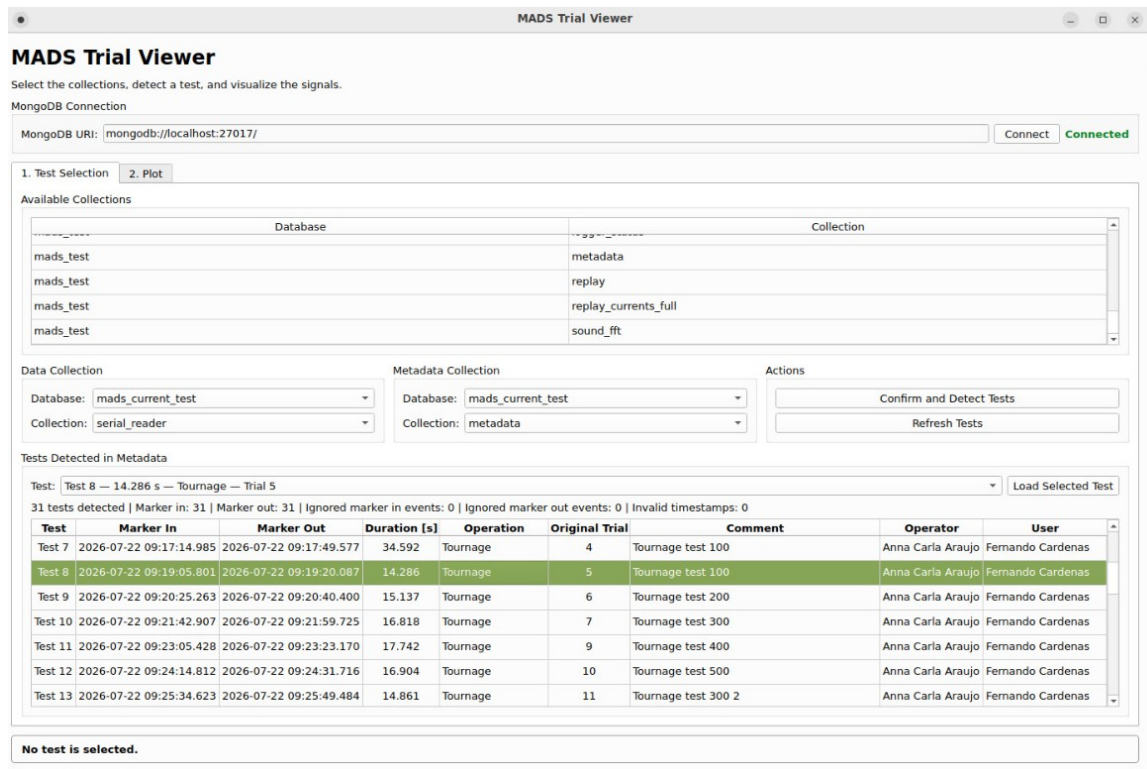


Figure 3: Main interface of the MADS Viewer used to select experimental data stored in MongoDB.

4.4. Implementation

The MADS Viewer was implemented in Python. PySide6 was used to build the graphical interface, PyQtGraph was used for interactive plotting and PyMongo was used to access the MongoDB database.

The selected data could be displayed simultaneously as a graph and a table. Basic statistical indicators were calculated to provide an initial summary of the selected interval.

The export functions allowed the user to generate:

- a CSV file containing the selected data points;
- a PNG image containing the displayed graph.

These functions simplify the transfer of experimental results to external analysis software and the preparation of figures for technical documentation.

5. Tests Performed

After commissioning the acquisition chain, measurements were recorded at five operating speeds of the machine. The objective was to verify that the complete system could acquire, transmit, store and display data under different operating conditions.

5.1. Experimental Procedure

Because the complete test sequence was carried out by a single operator, the acquisition and machine-operation steps had to be performed in a fixed and repeatable order. Before beginning the experiments, MADS and the complete measurement setup were activated and their operation was verified.

For each test, the metadata fields were first updated through the MADS TUI to identify the corresponding operating condition. The logger was then activated, followed by a mark-in command defining the beginning of the relevant recording interval. The machine was subsequently started and operated at the selected speed.

After the required acquisition period, the machine was stopped and a mark-out command was entered through the TUI to define the end of the test interval. The logger was then deactivated to prevent unnecessary data from being stored. Once these steps had been completed, the metadata were updated for the following operating condition and the same procedure was repeated.

The sequence applied to each experiment was therefore:

1. update the test metadata through the MADS TUI;
2. activate the logger;
3. record the mark-in timestamp;
4. start the machine at the selected speed;
5. maintain the operating condition during the acquisition interval;
6. stop the machine;
7. record the mark-out timestamp;
8. deactivate the logger;
9. prepare the metadata for the following test.

Five tests were performed at nominal rotational speeds of 100, 200, 300, 400 and 500 rpm.

The mark-in and mark-out timestamps, together with the metadata associated with each speed, were later used to identify and extract the corresponding measurement documents from the MongoDB collection.

5.2. Recorded Data

The five operating conditions were successfully recorded in the database. This confirmed that the acquisition chain could be used repeatedly and that the data corresponding to different experiments could be stored and accessed independently.

The MADS Viewer was used to retrieve the documents associated with each test using the corresponding mark-in and mark-out timestamps. The current measurements were then plotted

using the Arduino time in microseconds as the temporal reference.

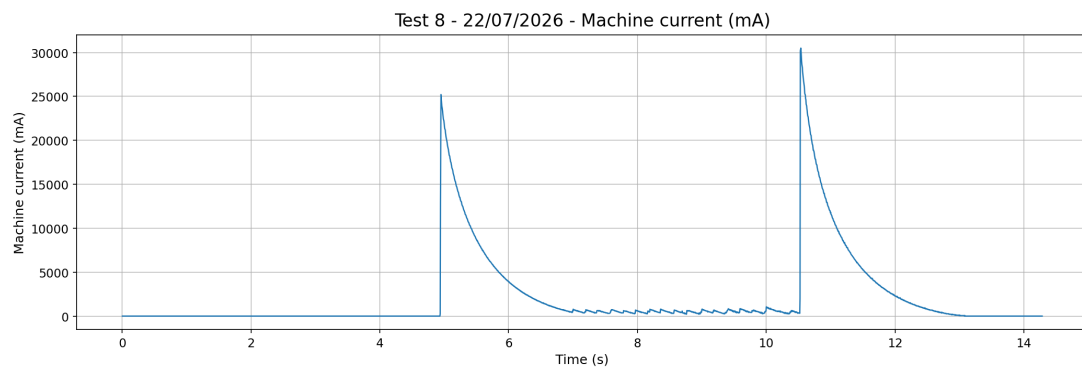


Figure 4: Current measurement recorded at 100 rpm.

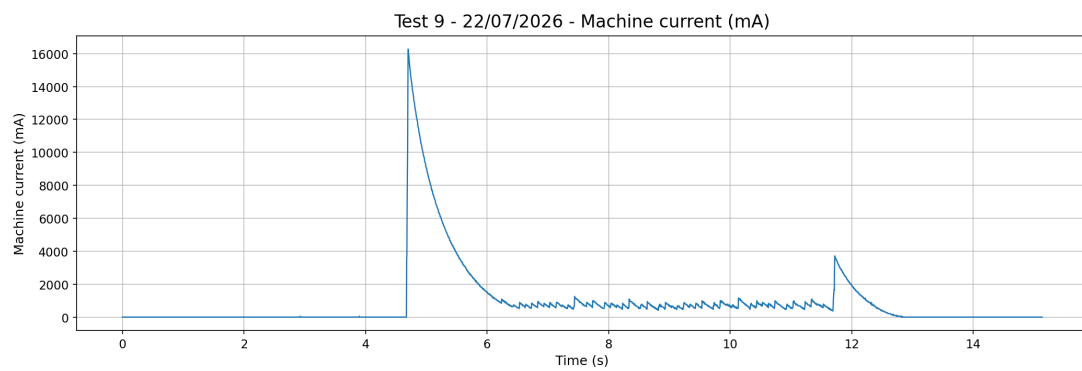


Figure 5: Current measurement recorded at 200 rpm.

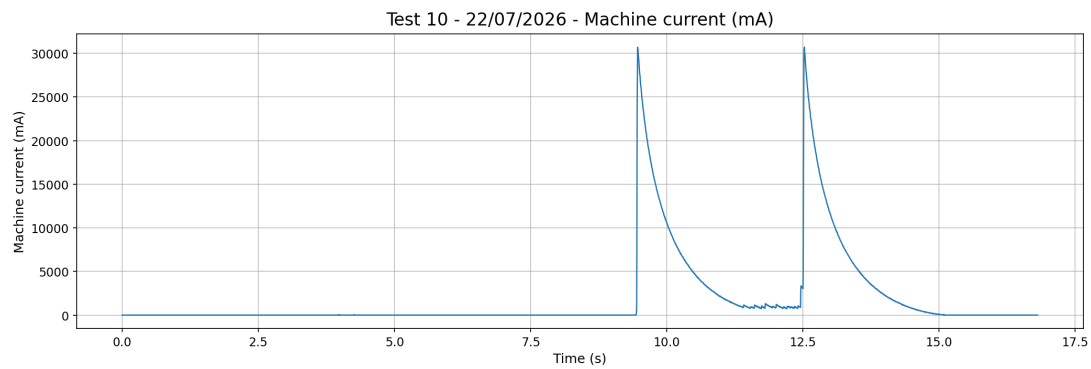


Figure 6: Current measurement recorded at 300 rpm.

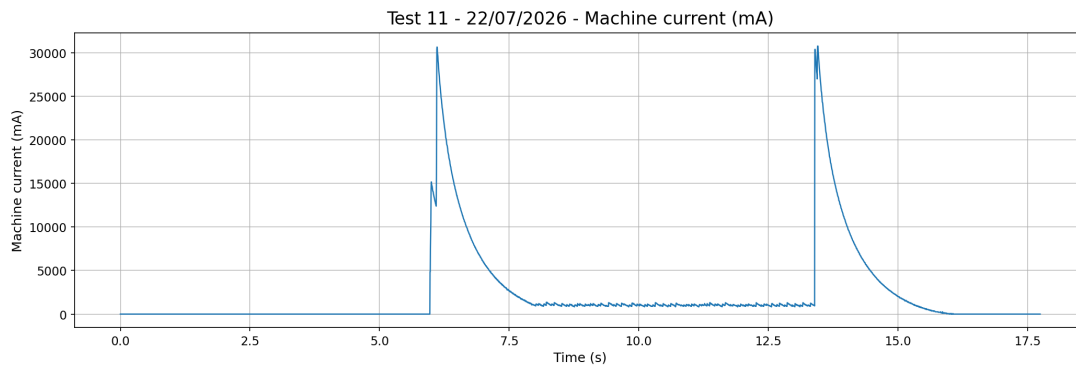


Figure 7: Current measurement recorded at 400 rpm.

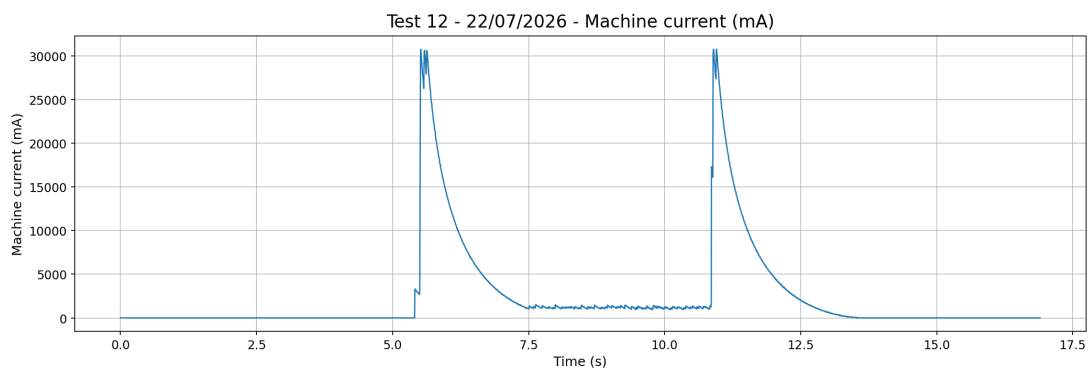


Figure 8: Current measurement recorded at 500 rpm.

Although the experiments were completed and the measurements were stored, the recorded data did not provide a sufficiently reliable distinction between the five operating speeds. This observation led to the evaluation of the current sensor described in the following section.

5.3. Limitations of the Signal Converter

In addition to the limitations associated with the current clamp, the investigation showed that the signal converter used during the first tests was not suitable for this application.

The current transformer produces an alternating signal whose amplitude is related to the current flowing through the measured conductor. To analyse this signal correctly, the conditioning circuit should preserve its temporal evolution so that the Arduino can acquire a representative sequence of samples.

The converter used during the tests included a rectification stage followed by a smoothing capacitor. In this configuration, the capacitor was charged when the rectified signal reached a new maximum and then discharged progressively through the circuit resistance. The converter therefore behaved as a peak-hold or envelope-detection circuit rather than as a transparent signal conditioner.

This behaviour introduced several problems:

- the output represented a retained peak instead of the instantaneous current waveform;
- the measured value depended on previous signal peaks;
- increases in current appeared rapidly, while decreases were delayed by the capacitor discharge

time;

- short variations and transient behaviour were smoothed or lost;
- measurements obtained at successive machine speeds could influence one another;
- the resulting signal could not be used to reconstruct a reliable temporal evolution of the current.

The discharge behaviour is governed approximately by the time constant

$$\tau = RC, \quad (1)$$

where (R) is the equivalent discharge resistance and (C) is the smoothing capacitance. A large time constant causes the converter output to retain the previous peak for a longer period. Consequently, even if the current consumed by the machine decreases, the measured voltage may remain temporarily close to the earlier maximum.

This explains why the recorded values did not react immediately to changes in the machine operating condition and why the five speed levels could not be compared reliably. The limitation was therefore not caused exclusively by the current clamp: the peak-retention behaviour of the converter also modified the measured signal.

It is important to distinguish this smoothing capacitor from the capacitor used in a DC-bias circuit. A DC-bias circuit does not rectify or retain the maximum value. Instead, it shifts the complete alternating waveform around half of the Arduino supply voltage, allowing both positive and negative parts of the original signal to be represented within the positive input range of the analog-to-digital converter.

6. Recommendations for the Measurement System

Based on the experimental results and the investigation of available measurement solutions, it is recommended to replace both the existing current clamp and its peak-hold converter.

The proposed measurement chain is:

SCT-013-060 current clamp $\longrightarrow$ AC signal conditioner $\longrightarrow$ Arduino analog input $\longrightarrow$ MADS
acquisition chain

6.1. Recommended Current Clamp: SCT-013-060

The recommended current clamp is the YHDC SCT-013-060. It is a non-invasive, split-core current transformer with a nominal input range from 0 to 60 A and a voltage output from 0 to 1 V.

This model is better suited to the application for the following reasons:

- its 60 A range is closer to the expected current levels than a higher-range sensor, provided that the maximum machine current remains below 60 A;
- the reduced measurement range should provide better use of the Arduino analog-to-digital converter resolution;
- it provides a voltage output and therefore includes its own burden resistor;

- it can be installed without interrupting the measured conductor;
- it is explicitly supported by the proposed LeChacal conditioning circuit.

The selected current range must be verified using a reference instrument before installation. The maximum instantaneous and RMS current must remain within the sensor rating under all machine operating conditions.

Product information and purchase link:

- [SCT-013-060, 60 A/1 V split-core current transformer](#)

6.2. Preferred Signal Conditioner: LeChacal CTC01

The preferred conditioning solution is the LeChacal CTC01 Current Transformer Conditioner. This board is designed to connect current transformers directly to an Arduino analog input.

Unlike the converter used during the initial tests, the CTC01 does not rely on a rectifier followed by a peak-hold capacitor. Instead, it preserves the alternating waveform and shifts its reference level to approximately half of the supply voltage. The Arduino can therefore sample the complete waveform and calculate the RMS current digitally.

The CTC01 is particularly suitable because:

- it explicitly supports voltage-output current transformers, including the SCT-013-060;
- no additional burden resistor is required with the SCT-013-060;
- it uses an AD623 instrumentation amplifier;
- its zero reference can be configured at $(V_{CC}/2)$;
- it can operate from either 5 V or 3.3 V;
- optional gain can be added to improve the ADC resolution at low current levels;
- the output waveform remains available for direct sampling and RMS calculation.

The gain must be selected carefully to prevent the signal from exceeding the Arduino ADC range. The complete system should subsequently be calibrated against a reference current measurement.

Documentation and purchase links:

- [LeChacal CTC01 technical documentation](#)
- [LeChacal CTC01 purchase page](#)

6.3. Alternative Integrated Solution: ElectroDragon Energy Monitoring Shield

An alternative solution is the Arduino Energy Monitoring Shield developed by ElectroDragon. This module provides dedicated inputs for YHDC non-invasive current transformers and supports up to three current-measurement channels.

Its main advantages are:

- direct mechanical integration with an Arduino board;
- three available current-sensor channels;

- compatibility with several YHDC current-transformer ranges;
- reduced external wiring compared with a breadboard implementation;
- possibility of extending the system to several measured conductors or phases.

Product and purchase link:

- [ElectroDragon Arduino Energy Monitoring Shield](#)

6.4. Alternative Custom Conditioning Circuit

The signal-conditioning electronics can also be assembled directly instead of using a commercial module. OpenEnergyMonitor provides a detailed description of how to interface a current transformer with an Arduino.

The proposed circuit adds a DC bias at half of the Arduino supply voltage. This allows the alternating sensor signal to oscillate above and below the bias value while always remaining inside the positive ADC input range.

For a 5 V Arduino, the zero-current level is placed near 2.5 V. The measured waveform then oscillates around this value. The Arduino software subtracts the offset and calculates the RMS value from a sequence of samples.

The capacitor used in this circuit has a different function from the capacitor in the rejected peak-hold converter. It stabilizes the midpoint reference and provides a low-impedance AC path; it is not placed after a rectifier to retain the signal maximum.

Because the SCT-013-060 is a voltage-output current transformer, an additional burden resistor should not be installed. The bias network and input protection must nevertheless be designed for the Arduino supply voltage and ADC limits.

Circuit design reference:

- [OpenEnergyMonitor: interfacing a CT sensor with an Arduino and adding a DC bias](#)

6.5. Recommended Implementation Sequence

The following sequence is recommended for continuing the experimental work:

1. verify the maximum current of the machine using a calibrated reference instrument;
2. confirm that the 60 A range of the SCT-013-060 is sufficient;
3. purchase the SCT-013-060 and the LeChacal CTC01 conditioner;
4. connect the CTC01 output to the Arduino analog input;
5. verify that the waveform is centred around ($V_{CC}/2$);
6. confirm that the waveform does not clip at 0 V or at the ADC reference voltage;
7. acquire several complete AC cycles at each machine speed;
8. calculate RMS current digitally from the raw samples;
9. calibrate the result against the reference instrument;
10. repeat the tests at 100, 200, 300, 400 and 500 rpm.

For the first validation campaign, the recommended combination is therefore the SCT-013-060 current clamp together with the LeChacal CTC01 conditioner. The ElectroDragon shield can be considered if a more integrated solution or several simultaneous measurement channels are required.

7. Continuation of the Work

The results obtained during the reporting period provide a clear basis for the remaining work. The next phase should preserve the functional acquisition and visualization chain while improving the quality and reliability of the physical measurement.

7.1. Improvement of the Measurement System

The highest-priority task is the selection of a more suitable current measurement solution. The expected current range and the amplitude of the variations between machine speeds must be estimated before selecting the new sensor.

Once selected, the new sensor should be integrated into the existing experimental structure. The complete measurement chain should then be calibrated using a reference instrument.

The following points should be verified:

- measurement range;
- sensitivity;
- resolution;
- bandwidth;
- offset and drift;
- electrical isolation requirements;
- influence of grounding and shielding;
- repeatability of the measurements.

7.2. Consolidation of the MADS Acquisition Chain

The MADS configuration should be consolidated for long-duration experiments. The stability of the source on the Raspberry Pi must be verified over extended recording periods.

The continuity of the sample number should be analyzed to identify missing or duplicated samples. The use of `t_us` as the experimental time reference should also be validated.

The final configuration should include clear documentation for:

- launching the required MADS agents;
- connecting the Raspberry Pi and the computer;
- configuring the broker address;
- starting the source;
- monitoring the experiment with the TUI;

- storing the measurements in the database.

7.3. Further Development of the MADS Viewer

The MADS Viewer can be improved to facilitate the comparison of future experiments. Possible developments include:

- comparison of multiple tests on the same graph;
- improved selection of time intervals;
- better handling of large database collections;
- clearer display of experiment metadata;
- additional statistical indicators;
- improved error messages;
- documentation of the installation and user workflow.

The CSV and PNG export functions should also be validated using the new experimental datasets.

7.4. Repetition of the Five-Speed Tests

After installation and calibration of the new measurement system, the five operating-speed tests should be repeated under controlled conditions.

Several repetitions should be performed for each speed to evaluate repeatability. The duration, loading condition and stabilization time should remain consistent between tests.

The new experimental campaign should make it possible to:

- compare the current consumption at the five speeds;
- quantify the variability of the measurements;
- evaluate repeatability;
- estimate measurement uncertainty;
- determine whether the operating speeds can be distinguished;
- validate the complete MADS acquisition and visualization workflow.

7.5. Proposed Work Plan

Table 5 summarizes the proposed continuation of the project.

Table 5: Proposed work plan for the remaining project period.

Task	Expected result	Priority	Status
Select a new sensor	Measurement range and sensitivity adapted to the application.	High	Planned
Calibrate the measurement chain	Reliable relationship between sensor output and current.	High	Planned
Validate long-duration acquisition	Stable source and database recording over extended periods.	High	Planned
Verify sample continuity	Quantification of missing or duplicated samples.	High	Planned
Improve the MADS Viewer	Faster comparison and export of experimental results.	Medium	In progress
Repeat the five-speed tests	Validated and repeatable dataset for all operating conditions.	High	Planned
Prepare technical documentation	Reproducible procedure for configuration, acquisition and analysis.	Medium	Planned

8. Conclusion

During the reporting period, the complete experimental acquisition chain was brought into operation. The physical measurement system was connected to the Raspberry Pi, and the resulting data were transmitted, published through MADS and stored in the database.

The practical use of MADS was studied through the execution of a source on the Raspberry Pi, communication between the Raspberry Pi and the computer, monitoring with the TUI, configuration through the `mads.ini` file and interpretation of the stored database documents.

The MADS Viewer was developed to simplify access to the experimental data. The application made it possible to select measurements from the database, display them graphically, inspect their numerical values and export the selected data and figures.

Tests were successfully performed at the five available operating speeds of the machine. These experiments validated the general acquisition, communication, storage and visualization workflow.

However, the tests also demonstrated that the selected current sensor was not suitable for reliably distinguishing the different operating conditions. The next stage of the project should therefore focus on selecting and calibrating a more appropriate sensor, consolidating the MADS acquisition chain and repeating the five-speed experiments under controlled conditions.